Follow Me Script

By
Velfire, July 2019

What it DOES

- Lead the pilot from the departure gate to the start of the runway (if travelling to the start of the runway does not require the car to drive on the runway itself),
- Lead the pilot from the runway to the arrival gate,
- Auto request for the Follow Me (FM) car (during departure) if the runway info could be read from FMC and the pre-requisites are met,
- Auto request for the FM car (during arrival) if the pre-requisites are met.

Features

- FM car automatically appears and lead the plane to the runway without user intervention (when the conditions are met)
- FM car automatically appears and lead the plane to the arrival gate without user intervention (when the conditions are met)
- Signals the pilot to slow down for a turn by indicating the direction of the turn
- Take a <u>shorter</u> route with lesser turns that may not be the <u>shortest</u> route

What it DOES NOT

- Allow user to specify a desired route,
- Show a map of the airport and your location on the map,
- Travel on the runway in the case where the plane needs to travel on a runway to get to the end for a u-turn and take off. The car will only lead the pilot to the entrance of the runway (known as the hot zone) and it's up to the pilot to navigate to the end of the runway for the u-turn and take off,
- Assign a gate that is definitely not occupied,
- Have a yellow and black checkered livery for the X-Plane car,
- Hide the Follow Me (i.e. FM) tab on the right-hand side of the screen,
- Assign the correct airport terminal and gates meant for that airline in the real world.

INSTALLATION

Prerequisite:

- X-Plane 11 (the script was developed and tested for X-Plane 11)

- Ensure you have FlyWithLua (FWL) plugin installed. If not, download and installed the **official NG (Next Generation) Edition of FlyWithLua for X-Plane 11**. Note that the FollowMe script was tested with version 2.7.17
- Airport with ATC Taxi Routes.

Installing the Follow Me Script:

- Unzip the FollowMe file and place the content in the FWL Script folder as shown below:
  - Place the .lua script in
    - X-Plane 11 > Resources > plugins > FlyWithLua > Scripts
  - Place the follow_me folder into
    - X-Plane 11 > Resources > plugins > FlyWithLua > Scripts

OPERATIONS

Open the User Interface Window

1. Click on the FM tab on the right-hand side of the screen to open up the User Interface (UI) window.
2. You may drag the FM tab vertically to reposition the UI window to your liking.

FM Tab

Current Airport

Departure Settings

Arrival Settings

Request for the Car and preference to limit its speed

Specify aircraft type

Various settings and saving the preferences

User Interface Window

Preferences

1. You may save the position of the FM tab. The position is regardless of the aircraft.
2. Besides the position of the FM tab, all other settings are tied to the aircraft.
3. When a new aircraft model is detected, default values are assigned initially.
4. Click on the “Save Pref.” button to save the settings for the specific aircraft model.
5. The preference file is saved in the preference folder of X-Plane with the filename “FollowMe.prf”

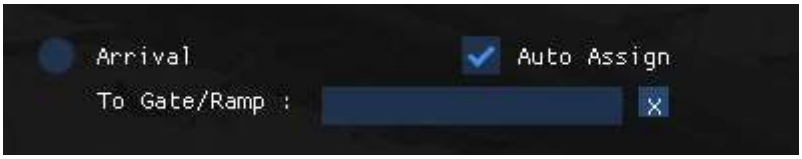
Specify the Aircraft Type

1. This information enables the plugin to determine the following:
    - Identify the gate(s) that can accommodate the aircraft
    - The taxiway and route suitable for the aircraft e.g. taxiway that can accommodate a A380
2. Whenever this info is changed, the list of arrival gate(s) will be re-determined to find the gate(s) that could accommodate the specified aircraft type. Likewise the route may be changed. As such, when the FM car is active (i.e. leading the pilot), the user is not allowed to change the aircraft type unless the request is cancelled.

Departure

1. The “Departure” mode is for the FM car to lead the pilot to a runway.
2. Specify the runway from the dropdown list.
   Note that only runways with ATC routes leading to it will be shown.
3. Click on the “Get from FMS” checkbox for the plugin to obtain the departure runway info from the FMS, relieving the user from the need to re-specify the runway info again.
4. When the plugin picks up that the plane is parked within the range of a gate, it would default the mode to “Departure”. The user may overwrite this and changed it to “Arrival”.

Arrival



1. The “Arrival” mode is for the FM car to lead the pilot from the runway exit to the designated arrival gate.
2. Specify the gate from the dropdown list.
3. The list of gate(s) will be filtered as you type. Click on the “X” to remove the filter.
4. Click on the “Auto Assign” checkbox to auto assign a gate when the plane touches down. This helps to relieve the pilot to focus on the cleaning up procedure instead of having to quickly call up the airport map and figure out which gate to go to, how to get to the gate and whether the gate/route taken can accommodate the plane.
5. If a suitable gate cannot be identified (e.g. no gate can accommodate a fighter jet), all restriction will be removed and all the gate(s) will the listed. The user will be informed when this happens.
6. When the plane landed, the mode be set to “Arrival”. The user may overwrite this and changed it to “Departure”.
7. When the plane landed and parked successfully (i.e. within the range) at the designated arrival gate, the mode will be automatically switched from “Arrival” back to “Departure”.
8. To help the pilot parked within the range of a gate, especially when the gate is not properly marked or it’s a tie-down ramp start, click on “Show Ramp Start”. A diamond object will be shown at the designated arrival gate. Park the plane facing the RED face of the diamond. The plane should be within range if you are using the autogate or SAM plugin to park the plane. If you are not or you are parking at a tie-down ramp start, the plane should stop when the diamond is right above the cockpit. You’ll know that you have parked within the range of a gate when you see name of the gate reflected besides the Departure row.

<u>Show Path</u>

1. Click on the “Show Path” checkbox if you would like to see the route that the FM car will be taking you.

<u>Show Ramp Start</u>

1. A diamond shape object will be shown
    a. at the departure gate when the mode is set to “Departure”
    b. at the arrival gate when the mode is set to “Arrival”
2. Park the plane by facing the RED face of the diamond.
3. When the plane is parked within 10m of the gate (i.e. the diamond object), the program will deem that the plane is parked at that gate. Beyond that, the program will assume that the plane is just passing by.

<u>Limit Speed to 20 kts</u>

1. Click on this checkbox if you want to FM car to travel within the 20kts limits. The car will stay within the 20kts limit even when the pilot pushes to car to go faster.
2. If this checkbox is not checked, watch your speed as you push the car. Observe the signal lights on the roof of the FM car. When the signal light is blinking in one direction only, it is telling the pilot that the FM car is turning to that direction. The pilot is to slow down or brake immediately for the turn.
3. For info, the X-Plane car chosen is a Ferrari 458 Speciale A model that can go up to 172 kts. The danger of pushing the car to go faster is that your plane may not be able to brake in time for a turn or come to a stop at the end point. Be prepared to crash!

<u>Conditions for Auto Request FM Car at Departure</u>

- "Get from FMS" checkbox is checked
- FMS function is available for the airplane
- Departure runway is specified in the FMS
- Either the Strobe light OR the Taxi light is switch on

Once the above conditions are met, the FM car will appear and the pilot will be told to "follow me".

<u>Conditions for Auto Request FM Car at Arrival</u>

- "Auto Assign" checkbox is checked
- Ground speed of the plane is less than 30 kts
- FM User Interface Window is closed
- Taxi light is switch on

Once the above conditions are met, the FM car will appear and the pilot will be told to "follow me".

When this does not happen, open the UI window and request the FM car manually.

<u>Vol</u>

1. Slide the bar to adjust the vol to your liking.
2. Click on the "Test Vol" button to play the sound while you adjust the volume.

<u>Understand the Different Ramp Start</u>

1. There are four types of Ramp Start defined in X-Plane namely Gate, Misc, Tie-Down and Hangar
2. If the ramp start is defined as a Gate or Misc, it is expected that a pushback is required when the plane departs. As such the FM car will appear behind the plane. However, if the

plugin is not able to determine that the plane is parked at a gate, it will be assumed that the plane is on the move and hence the FM car will appear in front of the plane

3. If the ramp start is defined as a Tie-Down or Hangar, the FM car will appear in front of the plane. If no route is found starting from the front, a route starting from behind the plane will be determined. In this case, the FM car will appear behind the plane.

Credit

1. Thanks to Markus Furrer for enhancing the code to enable the use of a typical follow me car and for the effort in asking the author of the apt_follow_me-01.obj from BS2001 Objects library for permission to modify and the use of the object.
2. Thanks to Paulmort for the consent to modify and use the follow me truck in this plugin.

Change Log

Version 1.0

Released Jul 2019

- Final Release

Version 1.1

Released Jul 2019

- Refine the path determination logic to handle runway crossing and in the scenario where there is no hotzone leading to the runway.

Version 1.2

Released Jul 2019

- Enhance the deciphering of the airport info in the apt.dat file especially in the case where there is only one airport info in the apt.dat file.
- In the runway dropdown list, show only runway(s) with ATC routes leading to it. This is to prevent user from selecting a runway that the program is unable to lead the way due to the absence of the ATC route.
- Refine the auto request for FM car at departure such that the FM car is deactivated without saying goodbye if the plane has already taken off.

- Refine the auto request for FM car at departure to check that the plane is on ground. This will prevent the case where auto request would be auto-triggered because the plane is flying when the script is loaded.
- Refine the auto request for FM car at arrival so that the FM car appears in front of the plane while the plane is moving.
- Refine the logic to better manage the path nodes in cases where the taxi nodes are defined very close to each other.
- Enable the auto determination of the screen width. To overcome the issue of a null value from the screen width variable when XP and the script is first loaded, wait for at least 1 sec after the script is loaded then get the screen width value.

Version 1.3

Released Oct 2019

- Inform user when there is no route leading to the runway specified in the FMS. Do not show runway that has no route leading to it.
- In the case of a tie-down or hangar ramp start, if a route could not be found starting from the front of the plane, the system will then look for a route behind the plane.
- Added two new follow me vehicle and the function to choose the type of follow me vehicle.
- Fix the issue of right steering wheel not working.

For Developers

<u>Impetus</u>

1. It took 5 months to complete this project, starting with learning to script in FlyWithLua, refreshing my knowledge of secondary school geometry and physics, understanding the C/C++ pointers in order to interact with the X-Plane SDK, learning to use Sketchup and modelling the Signal Light Bar and understanding the different file specification like the APT.DAT and the OBJ8 file specs.
2. It started off with the desire to focus on the cleaning up procedure upon landing instead of having to quickly call up the airport map and figure out which gate to go to, how to get to the gate and whether the gate/route taken can accommodate the plane. Likewise, when departing from the gate.
3. As the development progressed, I realised that people were having the similar queries as I do on how to program certain function(s). But very often these queries were either not answered or they were answered very vaguely. With those vague answers, I had spent quite a substantial effort and time on trial and error to make the function work. At times, the completion of the project hinges on that particular function to work e.g. interacting with the dataref in the object to animate the component.

4. I was determined to complete this project and make this script available to all so that others could refer to this script for code samples and as a working example to all those queries out there similar to what I had.
5. This plugin is a contribution to the X-Plane community as an appreciation to the countless kind souls out there who have put in countless effort and time to offer the many freeware to the X-plane community, enriching the flight experiences of many including myself.

Design Considerations

1. I chose FlyWithLua to program the plugin as the syntax is quite easy to understand and there are a lot of examples available for beginners like me to reference. C/C++ was too complicated for me and I have a big headache trying to understand pointers and the many data types.
2. I started off with making use of the ground routes and parking definition files generated by WT3. Near completion, I dropped everything as I wouldn't want the plugin to be dependent on WT3, and also the worry of copyright infringement.
3. The same approach for the FM car. For the script to be available for all to use, reference, and improved upon, I dropped the use of a third-party FM car and leveraged on the X-Plane car (which is why it does not have a yellow and black checkered livery) and placed a custom built light bar object on the roof of the car. I hope this approach does not infringe on any copyright. If it does, I'll be glad to remove anything of concern immediately. I've thought of modelling the FM car myself, but it's too much effort. I would like to get back to flying.
4. When I decided to make use of the APT.DAT file, it worries me that the Global Airport file was huge and it took quite some time to load using NotePad/WordPad. Hence the use of BUFSIZE to read a chunk and string.find to look for the specific pattern/string. The time taken seems acceptable. Most of the time the loading time is unnoticeable, in some cases just a slight freeze.
5. I took quite a long time figuring out how to program the car movement e.g. when should the steering wheel start to turn and when to straighten, what should be the radius of turn, when should the car slow down, how should the car behave when the plane is out of sight, what if the plane overtakes the car, what to do when the turn is more than 90 deg, what if a u-turn is required. Important points are
    a. Use the speed and acceleration formula to control the car movement
    b. The car (i.e. car heading) rotate w.r.t its rear wheel, not the reference axis of the car. The car's heading is 90 deg to the angle of rotation. I started off with using the reference axis of the car to turn the car and the tire is 90 deg to the angle of rotation while adjusting the heading of the car. The turn looks awkward.
    c. The plane is able to push the car as the program constantly track the distance between the plane and the car; accelerating when the distance is shortened, decelerating when the distance is too wide and coming to a stop when the plane is out of range. It is like a magnetic force, repelling and attracting.

6. With all the considerations above, I started to appreciate all the final tips mentioned in the article on "ATC Taxi Route Authoring". There is no need to "micromanage" the taxi routes. Define the minimum and let the AI program determine how it wants to move the vehicle.
7. For determining the route, I started with using Depth First Search because this is what I understand best. End up the search could never complete, especially at a big airport. Once the path heads out in a wrong direction in the early stage of path finding, due to its depth first objective, it'll take a very long time till it backs up to the point where it should have taken the other direction. The user could wait endlessly. Hence the whole logic was rewritten to use the A* Search logic. The path finding was super fast and accurate. In order to ensure that the logic could also minimise the number of turn, certain weightage (aka cost) is given for different degree of turn with the max given for a U turn. Hence, when the A* search looks for the next node to advance, it'll treat all nodes within a specific range to the shortest node as equal and would take the node with the minimum turns.
8. You'll notice that when you use the imgui function to design a window with a push button, it requires two clicks to push the button. First click will bring the window into focus and the second click will push the button. To achieve a one click push button, actively track the position of the mouse and use "float_wnd_bring_to_front (*wnd_handle*)" to bring the window into focus when the mouse is over the region of the window. So when the user click on the push button, one click is required as the window is already in focus when the mouse pointer is on it.

<u>Queries that the script answered and working example to achieve the following functions:</u>

1. How a FlyWithLua script interact with the C/C++ API with the use of FFI
2. How the C/C++ pointers, enumerated type and structure can be represented in LUA
3. The X-Plane 11 Instanced Drawing programmed using LUA
4. An example of how to read huge file using BUFSIZE. Need to understand that reading a chunk could break the line in the middle and hence to need to read the chunk plus a additional line.
5. Using XPLMProbeTerrainXYZ, there is a need to probe and re-probe to get a more accurate coordinate. Why the need to probe and re-probe is something I couldn't understand. Just do it.
6. Programming the A* Search.
7. Using FlyWithLua script to:
    a. Display, position and move an object in X-Plane
    b. Use of XPLMProbeTerrainXYZ
    c. Define a custom dataref, link the dataref to the object and pass the value to the dataref to animate the component of an object

End Note

1. Although the code works, my understanding could be wrong/incomplete. For example, I still do not understand C/C++ pointers. I only know that whenever I see XXX*, the asterisk means its pointer/address and I need to create an array in LUA in order to have an address that could be pass to the pointer variable in C/C++
2. I still struggled with passing the Dataref array that the X-Plane car required for its wheels. After a lot of trial and error, the code in the script is what works, but most likely that is not the way to code it. Apologies for that, but as long as the cat catches the mice, it doesn't matter if it's black or white.

References

There are many references I used, but the following are the crucial ones:

(1) Understanding Car Physics and the Formulas
- https://www.slideshare.net/hebronashraf/vehicle-rollover-analysis
- http://www.amsi.org.au/teacher_modules/pdfs/Maths_delivers/Braking5.pdf
- https://www.johannes-strommer.com/diverses/pages-in-english/stopping-distance-acceleration-speed/
- https://framesynthesis.com/drivingsimulator/2d/

(2) Path Finding, the A* Search
- https://www.youtube.com/watch?v=eSOJ3ARN5FM, Graph Data Structure 6. The A* Pathfinding Algorithm by Computer Science

(3) Understanding of programming languages and an online platform for testing code snippets e.g. formulating the string patterns, understanding pointers/address and programming the search algorithms
- https://www.tutorialspoint.com/lua/, for LUA programming
- https://www.tutorialspoint.com/cprogramming/, for understanding of C
- https://www.tutorialspoint.com/cplusplus/, for understanding of C++

(4) X-Plane SDK Documentation

(5) FlyWithLua Documentation and all the wonderful example scripts delivered with FlyWithLua

License


```
Copyright (c) 2019, Velfire

Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files (the "Software"),
to deal in the Software without restriction, including without limitation
```

the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.